

Тема 1. Класове. Капсулация. Член-функции. Конструктори. Деструктори.

Съдържание

- Клас
- Интерфейс на клас
- Конструктор по подразбиране
- Капсулация
- Член-функции
- Конструктор
- Деструктор
- Видове конструктори

Клас

- Класовете са потребителски дефинирани типове данни.
 - *Позволяват представяне (имплементиране в програма) на различни обекти.*
- Класовете включват:
 - *член-данни*
 - *член-функции*
- Членовете на класа могат да бъдат:
 - *частни*
 - *публични*
- Публичните членове на класа съставят неговия **публичен интерфейс**, който представлява набор от член-функции и член-данни, чрез които обектите на класа могат да бъдат **използвани извън класа.**

Дефиниция на клас

```
class Student {  
    public:  
        Student();  
        void read();  
        bool is_better_than (Student s) const;  
        void print() const;  
    private:  
        data fields;  
} best;
```

```
class ClassName{  
    public:  
        //декларация на конструктори  
        //декларация на член-функции  
    private:  
        //декларация на член-данни  
} име_на_обект(и);
```

Интерфейс на клас

- Извън класа публичните членове се ползват през името на обект:

<име на обект>.<име на член>

или през указател към обект:

<указател> -> <член>

- За да се **дефинира** клас трябва първо да се определи неговият публичен интерфейс.

Пример за интерфейс на клас

```
class Student{
public:
    Student(); //конструктор по подразиране
    Student(char*, double); //конструктор с параметри
    void read(); //мутатор
    void setName(char * name); //мутатор
    void setGrade(double grade); //мутатор
    bool is_better_than(Student s) const; //аксесор
    void print() const; //аксесор
    char * getName() const; //аксесор
    double getGrade() const; //аксесор
private:
    char * name; //член-данна
    double grade; //член-данна
};
```

Пример за интерфейс на клас

- Препоръчва се член-данните да се декларират в нарастващ ред по броя на байтовете, необходим за представянето им в паметта.
 - *Получава се оптимално изравняване до дума.*
- Типът на член-данна на клас не може да съвпада с името на класа, но типът на член-функция на клас може да съвпада с името на класа.
- Някои декларации на членове могат да бъдат предшествани от спецификаторите за достъп **private**, **public** или **protected**.
 - *Областта на един спецификатор за достъп започва от спецификатора и продължава до следващия спецификатор.*
 - *Подразбиращ се спецификатор за достъп е **private**. Един и същ спецификатор за достъп може да се използва повече от веднъж в декларация на клас.*

Пример за интерфейс на клас

- Препоръчва се, ако секцията **public** съществува, то тя да бъде първа в декларацията, а секцията **private** да бъде последна в тялото на класа.
- Достъпът до членовете на класовете може да се разгледа на следните две нива:
 - По отношение на член-функциите в класа е в сила, че те имат достъп до всички членове на класа. При това не е необходимо тези компоненти да се предават като параметри. Този режим на достъп се нарича **режим на пряк достъп**.
 - По отношение на функциите, които са външни за класа, режимът на достъп се определя от начина на деклариране на членовете.
 - Членовете на даден клас, декларирани като **private** са достъпни само в рамките на класа. Външните функции нямат достъп до тях.

Член-функции

- Базирайки се на действията, които може да се извършат върху член-данните член-функциите биват:
 - **Конструктори** – специални член-функции, чрез които се създават и се инициализират нови обекти.
 - **Мутатори** – променят данните на обекта: `best.read();`
 - **Аксесори** (член-функции за достъп) - не променят стойностите на данните от обекта (дефинират се като константни **const**): `best.print();`
 - Други функции/методи

Конструктори

- Дейности при създаване на обекти:
 - отделяне на памет;
 - задаване на начални стойности на член-данните;
 - запомняне на текущо състояние.
- **Инициализиране на обекти** – изпълнява се от специални член-функции наречени **конструктори**.
- Конструкторите са член-функции на класа, които:
 - имат също име, като името на класа;
 - не връщат стойност;
 - нямат тип данни;
 - Обикновено се използват за инициализиране на член-данните на класа.
- Всеки клас трябва да има поне един или повече конструктори.

Конструктор по подразбиране

- Конструктор без параметри се нарича **конструктор по подразбиране**:

Student();

Декларация

```
Student::Student() {  
    name = new char [2];  
    name = " ";  
    grade = 2;  
}
```

Дефиниция

- Извикване на конструктор по подразбиране: Student best;

Предефинирани конструктори

- C++ позволява да имаме дефинирани няколко функции с едно и също име, стига те да се различават по списъка си с параметри (предефиниране на функции).
- Един клас може да има няколко конструктора, като всички са с името на класа, но се различават по списъка си с параметри (броя и/или типа на параметрите)
- Конструкторите с параметри могат да приемат параметри за всяка една член-дана на класа или за част от тях
- Пример:

`Student()`

и

`Student(char *, double)`

Предефинирани конструктори

```
Student::Student(){  
    name = new char[2];  
    name = " ";  
    grade = 2;  
}  
Student::Student(char * n, double g){  
    name = new char[strlen(n)+1];  
    strcpy(name, n);  
    if(g>=2 && g<=6)  
        grade = g;  
    else if(g<2)  
        grade = 2;  
    else  
        grade = 6;  
}
```

Деструктор

- При разрушаването на обекти се извършват **заклучителни действия** – освобождаване на динамично заета памет, затваряне на файлове и пр.
 - *Противоположни на ефекта на инициализация*
- Автоматизацията на заключителните действия се осъществява чрез **деструкторите**.
- Деструкторът е специална член-функция, за която:
 - *името ѝ съвпада с името на класа*
 - *името ѝ предшества от знака тилда (~)*
 - *няма тип на връщан резултат*
 - *не приема параметри*
 - *извиква се автоматично*

Деструктор

- Извиква се при:
 - Излизане от блок, в който е бил създаден обект от клас
 - Разрушаване на обект чрез операторите *delete* и *delete[]*

- Ако в класа липсва декларация на деструктор, то от компилатора се създава деструктор по подразбиране.

- Пример:

```
Student::~~Student(){  
    delete [] name;  
}
```

Дефиниция на член-функции

- Дефиницията е аналогична на дефиницията на обикновена функция, но името на член-функцията се предшества от името на класа, на който тя принадлежи, следвано от **оператор за принадлежност** :: (оператор за област на действие)

- Пример:

```
void Student::print() const{  
    cout << "Име на студент: " << name << endl;  
    cout << "Среден успех на студент: " << grade << endl;  
}
```

- Функцията `Student::print()` ще отпечата член-данните `name` и `grade` за конкретен обект. Извиква се през име на обект:

Име на обект

`best.print();`

Член-функция

Параметри на член-функция

- В кода на член-функцията обектът не се споменава, т.е. обектът се явява **имплицитен** (неявен) параметър на член-функцията.
 - При компиляцията функцията получава този неявен параметър като указател към обекта, през който е извикана
- Параметър, който изрично е зададен на функция, както е параметърът **s** във функцията **is_better_than**, се нарича **експлицитен** (явен) параметър:

`bool is_better_than(Student s) const;`

!!! Всяка член-функция има точно един имплицитен параметър и 0 или повече експлицитни параметри

Дефиниция на член-функция

```
bool Student::is_better_than(Student s) const{  
    if(grade > s.grade) return true;  
    return false;  
}
```

Имплицитен параметър

Експлицитен параметър

`if(next.is_better_than(best))...`

- Къде се записват дефинициите на член-функции?
 - Веднага след декларацията на класа, на който са членове.
 - При разделно компилиране се дефинират в съответния сорс файл описващ реализацията на заглавния файл.
 - В тялото на класа, при което се компилират като вградени (*inline*).

Вградени член-функции

- *Цел на вградените член-функции:*
повишаване
бързодействието на
програмите, тъй
като кода на
вградените
функции не се
съхранява на едно
място, а се копира
на всяко място в
паметта на
програмата, където
има обръщение
към функцията

- *Пример:*
`class Point{`
`public:`

Това **inline**
е излишно

```
Point() {x = 0; y = 0;}  
Point(int a, int b)  
{ x = a; y = b; }  
inline int get_x() const  
{ return x; }  
int get_y() const  
{ return y; }  
void print() const  
{ cout << x << ", " << y; }  
  
private:  
int x, y;
```

```
};
```

Всички дефинирани
по този начин член-
функции на класа
Point са **inline**

Клас Student

- Пълно описание на програмата за сравнение на студенти:

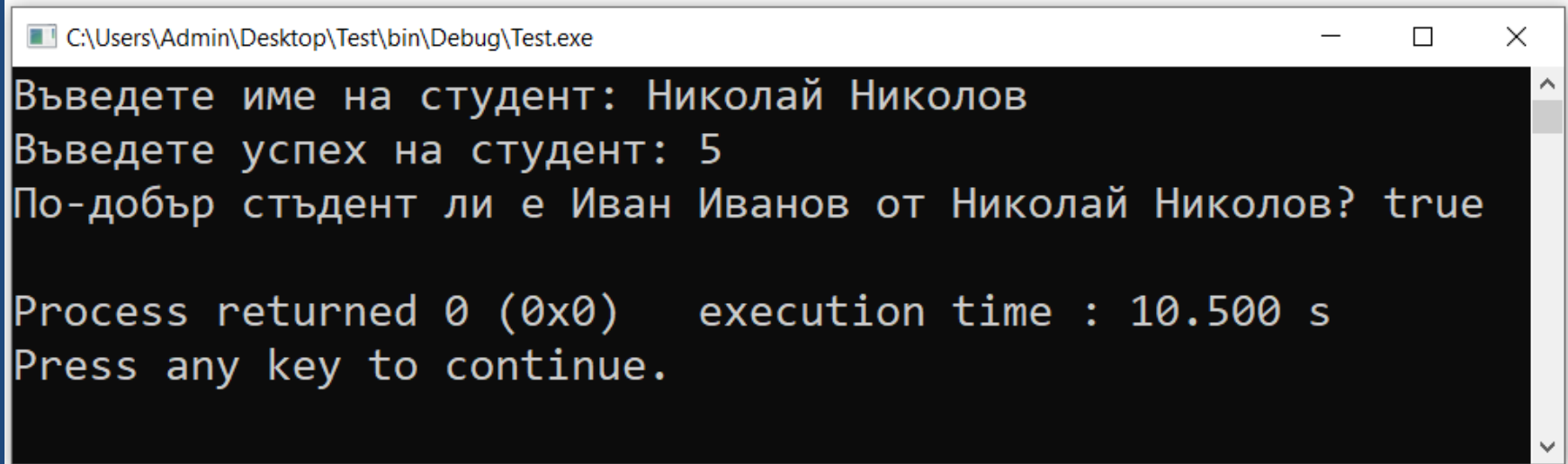
```
//student.h

#ifndef _STUDENT_H
#define _STUDENT_H

class Student{
public:
    Student(); //конструктор по подразибране
    Student(char*, double); //конструктор с параметри
    void read(); //мутатор
    void setName(char * name);
    void setGrade(double grade);
    bool is_better_than(Student s) const; //аксесор
    void print() const;
    char * getName() const;
    double getGrade() const;
```

Клас Student

■ Изход:



```
C:\Users\Admin\Desktop\Test\bin\Debug\Test.exe
Въведете име на студент: Николай Николов
Въведете успех на студент: 5
По-добър студент ли е Иван Иванов от Николай Николов? true

Process returned 0 (0x0)    execution time : 10.500 s
Press any key to continue.
```

Капсулация

- **Състояние на обекта** - всеки обект на клас съхранява конкретни стойности (посредством член-данните на класа), които се задават чрез конструктора на класа и могат да се променят чрез мутаторите.
- Действието, при което се скриват детайли от имплементацията на класа се нарича **капсулация**. Имплементира се чрез следните спецификатори за достъп:
 - *private*
 - *public*
 - *protected*
- Член-данните са част от класа, които обичайно не са видими за потребителите на класа (*private*). Те са скрити.

Капсулация

- Капсулацията се извършва чрез следните спецификатори за достъп:
 - *private* – членовете на един клас са достъпни само от други членове на същия клас или приятели на класа;
 - *protected* – членовете са достъпни от други членове на същия клас и от членове на класове, които наследяват този клас или приятели на класа;
 - *public* – членовете са достъпни навсякъде, където обект от този клас е видим;
- Обявените като „приятели“ (*friend*) функции и други класове имат достъп до всички членове на класа.
 - *Нарушава принципа за капсулиране, но създава удобства при взаимодействие между класове.*

Капсулация

Пример:

```
int main(){  
    ...  
    cout << best.name;  
    //грешка - не можем да достъпваме private елемент  
    //извън рамките на класа  
    ...  
}
```

- **Целта на капсулацията** е да защити и запази целостта на обекта.

Капсулация

Пример:

```
Student best(„Иван Иванов“, 4.89);
```

```
best.grade += 2; //!!!Проблем - ще получим успех 6.89
```

Решение:

Функцията

```
void setGrade(double grade);
```

ще се погрижи успехът на студента винаги да е стойност от 2 до 6

Извикване на конструктори

```
int main(){
```

```
    Student joe;
```

Извиква се
конструктора по
подразбиране

```
    Student ivan("Иван Иванов", 5.32);
```

```
    return 0;
```

```
}
```

Извиква се конструктора с
параметри
Student(char *, double)

Извикване на конструктори

- В определени случаи един конструктор е необходимо да извика друг конструктор, за да успее да инициализира член-данните си
- Пример:

```
class Student{
public:
    Student(); //конструктор по подразибране
    Student(char*, double); //конструктор с параметри
    Student(char *, double, int , int);
    void read(); //мутатор
    void setName(char * name);
    void setGrade(double grade);
    bool is_better_than(Student s) const;
    void print() const;
    char * getName() const;
    double getGrade() const;
private:
```

Извикване на конструктори

- Опит за изчистване на обект чрез извикване на конструктор

```
Student ivan("иван Иванов", 4.58);
```

...

```
ivan.Student(); // Грешно
```

Конструкторите се извикват само при създаване на обект

```
ivan = Student(); // Правилно
```

Създава се нов обект, който е празен и този обект се присвоява на **ivan**

Конструктори с подразбиращи се параметри

```
#include <iostream>
#include <windows.h>
#include <locale>
#include <iostream>
```

```
using namespace std;
```

```
class Point{
public:
```

Конструкторът
Point(int =0, int =0)
е както конструктор с
параметри, така и
конструктор по
подразбиране

- C++ поддържа функции с подразбиращи се параметри, като за тях се задават подразбиращи се стойности.
- Подразбиращите се стойности се използват само ако при извикването на функцията не се зададе стойност.
- Задаване на подразбираща се стойност за параметър, се извършва чрез включване на конкретна стойност за параметъра в прототипа на конструктора или в заглавието на неговата дефиниция.

Конструктори с подразбиращи се параметри

```
#include <iostream>
#include <windows.h>
#include <locale>
#include <iostream>
```

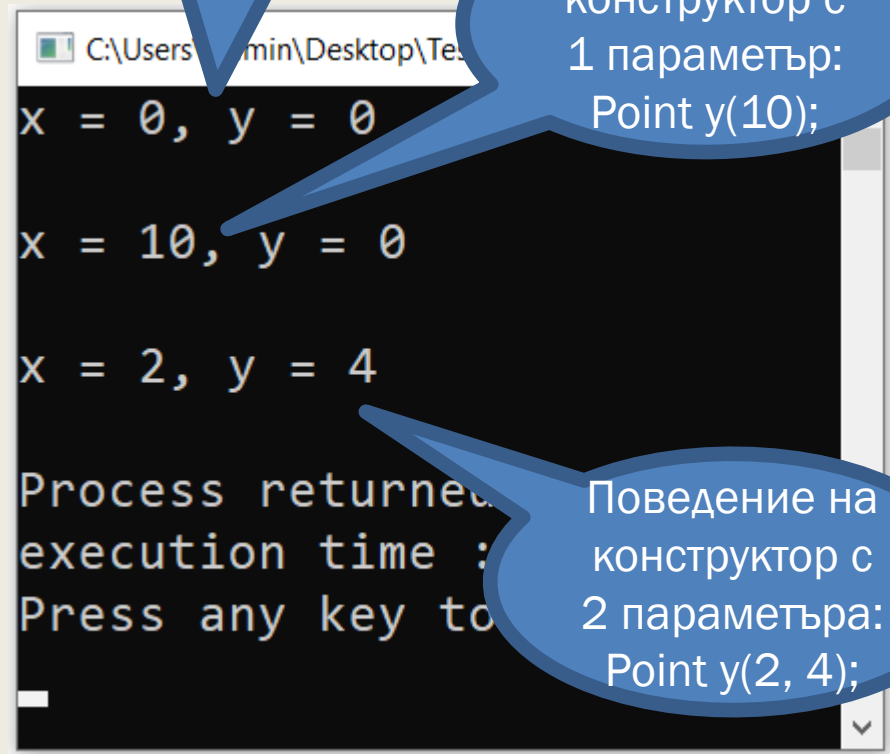
```
using namespace std;
```

```
class Point{
public:
```

Конструкторът
Point(int =0, int =0)
е както конструктор с
параметри, така и
конструктор по
подразбиране

Поведение на
конструктор по
подразбиране:
Point x;

Поведение на
конструктор с
1 параметър:
Point y(10);



```
C:\Users\min\Desktop\Tes
x = 0, y = 0
x = 10, y = 0
x = 2, y = 4
Process returned
execution time :
Press any key to
```

Поведение на
конструктор с
2 параметъра:
Point y(2, 4);

Конструктори с подразбиращи се параметри

- При използване на подразбиращи се параметри, важна роля играе редът на параметрите.
 - *Прието е, че ако параметър на функция/конструктор е подразбиращ се, всички параметри след него също са подразбиращи се.*
- Ако за даден подразбиращ се параметър е зададена стойност при обръщението към функцията/конструктора, за всички параметри пред него също трябва да са указани такива.
- Инициализацията на новосъздаден обект на даден клас може да зависи от друг обект на същия клас. За да се укаже такава зависимост се използва знакът за присвояване или кръгли скоби.

Конструктори с инициализиращ списък

- Цел – да се създаде конструктор, който инициализира член-данните преди тялото на конструктора
- Пример:

```
Point::Point(int x, int y): x(x), y(y) { }
```

Конструктор за присвояване и копиране

- При копиране на обекти се извършва по битово копиране на съдържанието на тялото на обекта на ново място в паметта.
- Случай, при които се налага копиране на обекти:
 - *Присвояване между еднотипни обекти*
 - *Предаване на обект като параметър по стойност във функция*

Копиране на обекти

- При присвояване копираната двоична последователност става ново съдържание на обекта, стоящ от ляво на операцията за присвояване. Този обект се деструктурира при **напускане на неговата област** на действие:

```
Point a;
```

```
Point b(4, 7);
```

```
a = b;
```

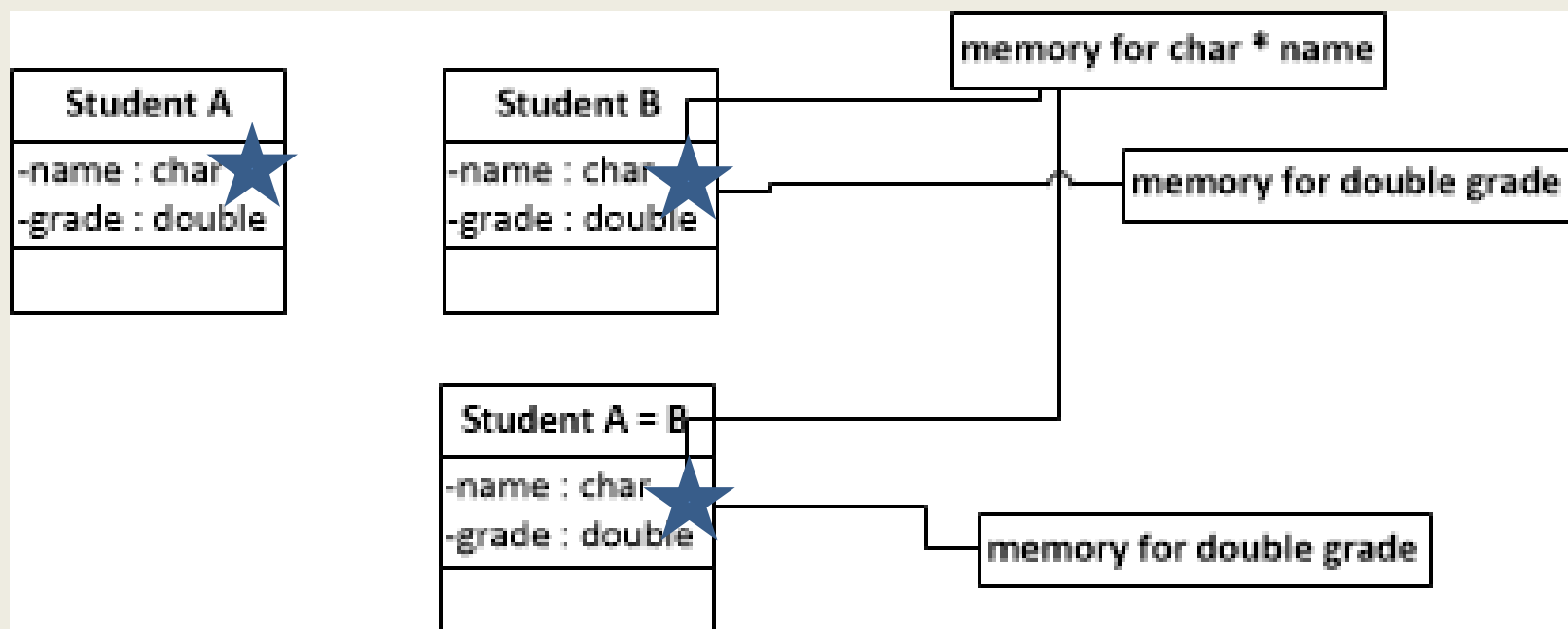
```
a.print();
```

```
b.print();
```

- При предаване на обект като фактически параметър по стойност, двоичното съдържание на обекта се копира в стековата рамка на извикваната функция. Това копие се деструктурира **автоматично и задължително** при излизане от функцията, преди разпадане на нейната стекова рамка

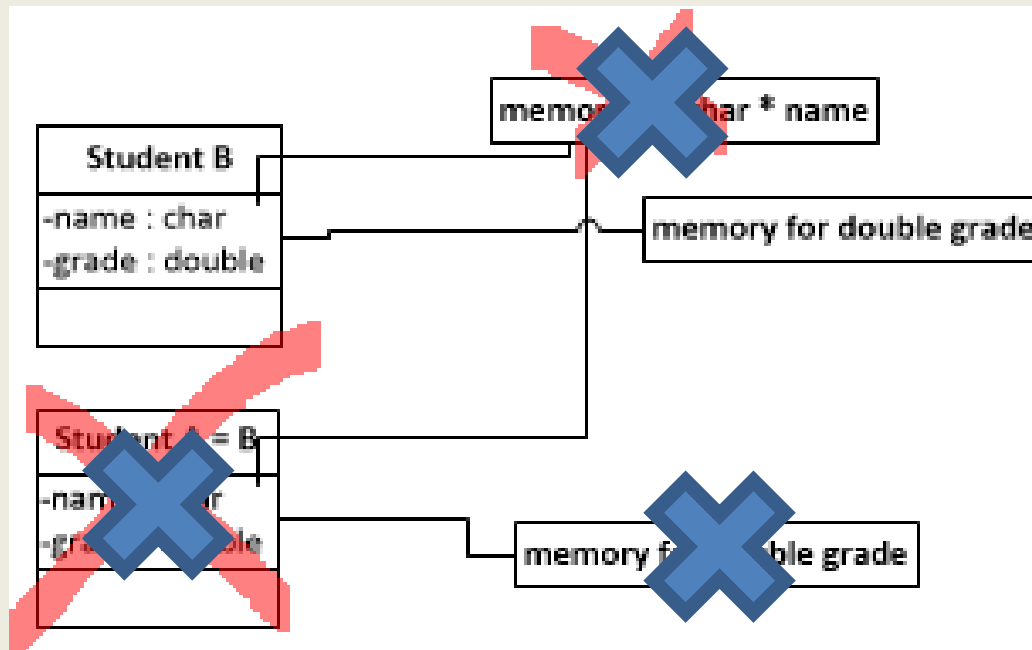
Копиране на обекти - проблеми

- При копиране ако копието споделя общи ресурси с оригинала (най-често при динамична памет), автоматичното извикване на деструктор за копиран обект, може да повреди оригинала

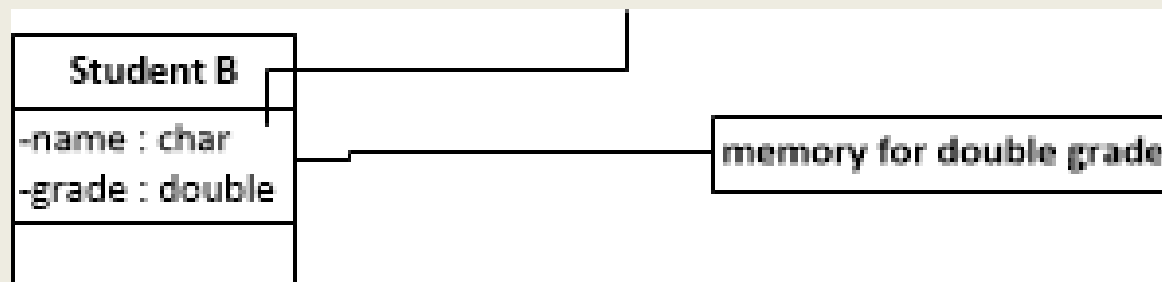


Копиране на обекти - проблеми

■ Извикване на деструктор за копието



■ Резултат – повреда на оригинала



Копиращ конструктор

- За избягване на по битово копиране на обекти в C++ е предвиден специален конструктор наречен **копиращ конструктор**:

- Синтаксис:

```
<име_на_клас>(const <име_на_клас> & <име_на_обект>){  
    <тяло>  
}
```

Приема като параметър обект от същия клас, предаван по адрес

- Пример:

```
Student (const Student & s){  
    name = new char[strlen(s.name)+1];  
    strcpy(name, s.name);  
    grade = s.grade;  
}
```

Копиращ конструктор

- **!!!Ако е дефиниран копиращ конструктор компилаторът *автоматично и задължително* го прилага вместо да копира бит по бит.**

- Пример за извикване на копиращ конструктор:

Student s1; // извиква се конструктор по подразбиране

// тук се извиква копиращ конструктор

// данните за обект s2 се копират от обект s1

Student s2 = s1;

// Също се извиква копиращ конструктор

// данните за обект s3 се копират от обект s1

Student s3(s1);

Указател *this*

- Всяка член-функция на клас поддържа допълнителен формален параметър, а именно указател с име *this*
 - *this* е от тип `<име_на_клас> *` за член-функциите, които са мутатори.
 - *this* е от тип `const <име_на_клас> *` за член-функциите за достъп

- Пример:

```
void Student::set_grade(double grade){  
    if(grade>=2 && grade<=6) this->grade = grade;  
    else if(grade<2) this->grade = 2;  
    else this->grade = 6;  
}
```

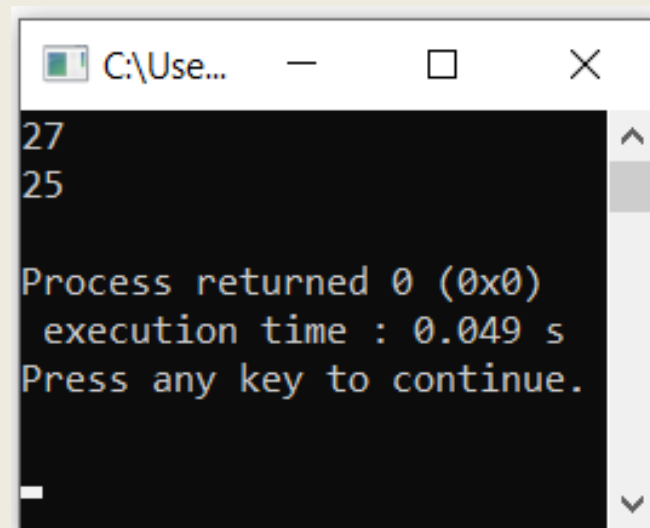
Ключова дума *explicit*

- Одноаргументните конструктори могат да се използват за неявно преобразуване на типа на обектите от един тип в друг.
- Това може да се ограничи и да се „заставят“ едноаргументните конструктори да се използват само за създаване на нови обекти, чрез ключовата дума *explicit*.
- Пример:

```
#include <iostream>

using namespace std;

class Int{
    int n;
public:
    explicit Int(unsigned up){
        n=rand()%up;
    }
};
```



```
C:\Use...
27
25
Process returned 0 (0x0)
execution time : 0.049 s
Press any key to continue.
```

Пример

- Клас представящ рационално число, включващ различните видове конструктори и деструктор.

```
//Rational.h

#ifndef _RATIONAL_H
#define _RATIONAL_H

class Rational{
public:
    //конструктор с подразбиращи се параметри,
    //може да бъде използван и като конструктор по подразбиране
    Rational(int =0, int =0);
    //копиращ конструктор
    Rational(const Rational &);
    //деструктор - тук нямаме нужда от такъв, но ще го ползваме с цел
    //проследяване освобождаването на обекти
    ~Rational();
    //функции мутатори - използват се за задаване на стойности на член-
    ланните на класа
```


Пример

- Клас представящ рационално число - Изход.

```
C:\Users\Admin\Desktop\Test\bin\Debug\Test.exe
r[1]=1/2
destruct number1/2
r[2]=2/3
destruct number2/3
r[3]=3/4
destruct number3/4
r[4]=4/5
destruct number4/5
r[5]=5/6
destruct number5/6
p=1/8
q=2/9
1/3
destruct number1/3
Знаменателят е равен на 0!
z=0/0
w=1/8
destruct number1/8
destruct number0/0
destruct number2/9
destruct number1/8

Process returned 0 (0x0)   execution time : 0.087 s
Press any key to continue.
```

Област на класовете

- За разлика от функциите, класовете могат да се декларират на различни нива в програмата:
 - глобално (на ниво функция)
 - локално (вътре във функция или в тялото на друг клас).
- Областта на глобално деклариран клас започва от декларацията и продължава до края на програмата.
 - Примерите досега бяха с такива класове.
- **!!!Ако клас е деклариран във функция, всички негови член-функции трябва да са вградени (*inline*). В противен случай ще се получат функции, дефинирани във функция, което не е ВЪЗМОЖНО.**

Област на класовете

■ Пример:

```
void f(int i, int* p) {  
    int k;  
    class CL {  
        public:  
            // всички методи са дефинирани в тялото на класа  
        private:  
            ...  
    };  
    // тяло на функцията f  
    CL x;  
    ...  
}
```

Област на класовете

- Областта на клас, дефиниран във функция, е функцията.
 - *Обектите на такъв клас са видими само в тялото на функцията.*
- Възможно е използването на обекти (в широкия смисъл на думата) с еднакви имена.
 - *В сила е правилото, че в областта си локалният обект скрива нелокалния.*
- Не е възможно в тялото на локално дефиниран клас да се използва функцията, в която класът е дефиниран.
 - *Пример: Нека сме в означенията на горния пример.*

```
void f(...) {  
    class cl {  
        // не може да се използва функцията f  
    };  
}
```

Въпроси и задачи

1. Какво представлява интерфейсьт на един клас? Какво е имплементация на клас?
2. Какво е член-функция и по какво се различава тя от обикновена функция?
3. Какво представлява функция мутатор? А функция за достъп?
4. Какво ще се случи ако забравим ключовата дума *const* в една функция за достъп? А какво ще се случи, ако по погрешка добавим думата *const* във функция мутатор?
5. Какво представляват имплицитните параметри? Как се различават те от експлицитните?

Въпроси и задачи

6. Колко имплицитни параметъра може да има една член-функция? А колко имплицитни параметри може да има една не-член-функция?
7. Колко експлицитни параметри може да има една функция?
8. Какво представлява конструктора и за какво се използва?
9. Какво е конструктор по подразбиране? Какво е последствието, ако един клас няма конструктор по подразбиране?
10. Колко конструктора може да има един клас? Можем ли да имаме клас без конструктор?
11. Ако един клас има повече от един конструктор, кой от тях се извиква?

Въпроси и задачи

12. Как можем да създадем обект на клас, който не е инициализиран чрез конструктор?
13. Как се декларира една член-функция? А как се дефинира?
14. Какво представлява капсулацията и с какво е полезна?
15. Член-данните на класа се дефинират в *private* секцията, за да се скрият, но как точно се скриват, при положение че всеки може да отвори класа и да ги види?
16. В коя секция се декларират конструкторите и защо? Можем ли да имаме конструктор на клас деклариран в *private* секцията? Ако да, какви са последствията за класа?
17. Може ли да имаме деструктор деклариран в *private* секцията на класа? Дайте обяснение.

Въпроси и задачи

18. За функции, които не са член на клас е лесно да определим ако в тях има извиквания на член-функции и извиквания на обикновени функции. Как ги различаваме? Защо не е така лесно да правим такава разлика, когато функция се извиква в тялото на член-функция?
19. Как определяме дали един имплицитен параметър се подава по стойност или по референция? А как определяме дали един експлицитен параметър се подава по стойност или по референция?
20. Можем ли да дефинираме указател към обект?
21. Можем ли да дефинираме масив от обекти? А динамичен масив от обекти?

Въпроси и задачи

- **Задача 1.** да се дефинира клас *Point*, определящ точка в тримерното пространство с реални координати, с три член-данни декартовите координати на точката и подходящи член-функции. Да се използва дефинираният клас, за да се напише програма, която въвежда n различни точки от равнината, след което:
 - ги мащабира по координатните оси x , y и z чрез мащабиращи множители съответно $a = -2$, $b = 0.5$, $c = 5$
 - извършва мащабиране по координатната ос y чрез мащабиращ множител $b = 0.25$ и трансляция чрез вектор $s1 = (0, 0, 4, 0)$

Благодаря за вниманието!